



Student Internship Portal

Project Report

Şerife Şevval Koç - 20220602055

Betül Özsan - 20210602050

Ayşe Serra Er - 20220601024

TABLE OF CONTENTS

Table of Contents.....	2
1. Introduction.....	3
1.1 Objectives.....	4
2. Requirements.....	4
2.1 Functional Requirements.....	4
2.2 Non-Functional Requirements.....	5
3. Database Schema.....	6
3.1 SQL Creation Scripts.....	6
Users.....	6
Student Profiles.....	6
Jobs.....	7
Applications.....	7
Notifications.....	7
4. LINQ Queries.....	7
4.1 Job Search with Skill, Location, and Type Filters.....	7
Code.....	7
Explanation.....	8
4.2 Admin Job Filters (Desktop).....	8
Code.....	8
Explanation.....	9
4.3 Home ViewModel — Highest Skill Match.....	9
Code.....	9
Explanation.....	9
5. Delegates and Events.....	9
5.1 NotificationManager — NotificationCreated Event.....	9
Purpose.....	9
Definition (Shared.Services.NotificationManager).....	9
Subscription in AdminDashboardForm.....	9
How it works.....	9
5.2 AuditLogger — AdminActionPerformed Event.....	10
Purpose.....	10
Definition (Shared.Utilities.AuditLogger).....	10

Subscription in AdminDashboardForm.....	10
How it works.....	10
6. MVC Structure.....	11
6.1 Overview.....	12
6.2 Service Layer.....	12
7. Sources.....	12
7.1 GitHub Repository.....	12
7.2 How to Run.....	12
Prerequisites.....	12
Steps.....	12
7.3 Video Demonstrations.....	13
8. User Interface and Design.....	13
8.1 Desktop Admin Application.....	13
8.2 Student Web Application.....	16
9. Unit Tests.....	20
9.1 Sample Test Scripts.....	21
Test 1 - CalculateSkillMatchPercentage_FullMatch_Returns100.....	21
Test 2 - CalculateSkillMatchPercentage_NoMatch_Returns0	21
Test 3 - CalculateSkillMatchPercentage_PartialMatch_ReturnsCorrectPercentage.....	22
Test 4 - CalculateSkillMatchPercentage_CaseInsensitivity_Returns100	22
Test 5 — CalculateSkillMatchPercentage_EmptyRequiredSkills_Returns0	22
Test 6 — CalculateSkillMatchPercentage_ExtraStudentSkills_Returns100	23
Test 7 — CalculateSkillMatchPercentage_TrimAndWhitespace_ReturnsCorrectPercentage	23
Test 8 — Contains_CaseInsensitiveSearch_ReturnsTrue	23
Test 9 — Contains_EmptyValue_ReturnsTrue	24
Test 10 — Contains_Mismatch_ReturnsFalse	24
10. Conclusion.....	25

1. Introduction

The Student Internship Portal is a dual-platform application designed to bridge the gap between students seeking internship and job opportunities and the administrative staff responsible for managing those listings. The system is composed of two tightly integrated components:

- A Desktop Admin Application (Windows Forms) that enables administrators to post and manage job listings, review and update student applications, generate summary reports, and monitor system activity through an audit log.
- A Student Web Application (ASP.NET Core MVC) that allows registered students to browse active job postings, filter by skill or location, apply for positions, track application statuses, and receive notifications.

Both applications share a common shared library for data models, database access, business services, and utility classes, ensuring consistency and reducing duplication across the solution.

1.1 Objectives

- Provide a centralised platform for students to discover and apply for internships and full-time roles.
- Enable administrators to manage the full lifecycle of job postings and applications.
- Deliver real-time notifications to students when their application status changes.
- Demonstrate advanced C# concepts including LINQ, delegates, events, and the MVC design pattern.
- Ensure data integrity and security through parameterised SQL queries and password hashing.

2. Requirements

2.1 Functional Requirements

Requirement	Description
User Registration & Authentication	Students and admins must be able to register, log in, and authenticate.
Student Profile Management	Students can create, update, and manage profiles including skills, education, and experience.
Resume Builder	The system allows students to generate and download resumes based on profile data
Job Posting Management (Admin)	Admins can create, edit, delete, and publish internship/job postings.
Job Search & Filtering	Students can search jobs using filters such as skills, location, and job type (using LINQ).
Job Matching System	The system automatically matches students to jobs based on their skills and profile data.

Requirement	Description
Application Submission	Students can apply for jobs and upload resumes.
Application Tracking	Students can track application status (e.g., pending, accepted, rejected).
Notifications System	Students receive notifications for new matches, application updates (implemented using delegates).
Admin Dashboard & Reporting	Admins can view analytics such as number of applications, active jobs, and user activity.

2.2 Non-Functional Requirements

Category	Requirement
Performance	The system should handle multiple users simultaneously within minimal time (< 500ms).
Scalability	The system must support increasing numbers of users, job postings, and applications without performance degradation.
Security	Sensitive data (passwords, personal info) must be encrypted and protected against unauthorized access.
Usability	The interface (web and desktop) should be intuitive.
Reliability & Availability	The system should ensure high uptime (e.g., 99%) and maintain data access via the shared database.

3. Database Schema

This project uses SQLite as its database management system. The database file is stored within the project directory at **Database/student_portal.db**. Instead of using Entity Framework, the project directly interacts with the database through raw SQL queries using the **Microsoft.Data.Sqlite** package.

Database connectivity is managed by the **DatabaseHelper** class located in **Shared/Data/DatabaseHelper.cs**. To improve portability and deployment flexibility, the database configuration was modified to use a relative path rather than an absolute path. As a result, the application can be executed on different computers without requiring any changes to the connection string. The database file is automatically resolved based on the project structure, making installation, distribution, and maintenance more convenient.

The application initializes the database automatically at startup through the **DatabaseInitializer** component, which creates the required tables if they do not already exist. The database schema consists of five main tables: **Users**, **StudentProfiles**, **Jobs**, **Applications**, and **Notifications**.

In addition, the system automatically creates a default administrator account during database initialization if no administrator account exists. This seeded account uses the username **admin** and password **admin123**, ensuring that administrative access is available when the application is first deployed.

3.1 SQL Creation Scripts

- **Users**

```
CREATE TABLE IF NOT EXISTS Users (  
    Id      INTEGER PRIMARY KEY AUTOINCREMENT,  
    Username TEXT NOT NULL UNIQUE,  
    PasswordHash TEXT NOT NULL,  
    Role    INTEGER NOT NULL DEFAULT 0, -- 0=Student, 1=Admin  
    CreatedAt TEXT NOT NULL  
);
```

- **Student Profiles**

```
CREATE TABLE IF NOT EXISTS StudentProfiles (  
    Id      INTEGER PRIMARY KEY AUTOINCREMENT,  
    UserId  INTEGER NOT NULL UNIQUE REFERENCES Users(Id),  
    FullName TEXT NOT NULL DEFAULT "",  
    Skills  TEXT NOT NULL DEFAULT "",  
    Education TEXT NOT NULL DEFAULT "",  
    Experience TEXT NOT NULL DEFAULT "",  
    Phone   TEXT NOT NULL DEFAULT "",  
    AboutMe TEXT NOT NULL DEFAULT ""  
);
```

- **Jobs**

```
CREATE TABLE IF NOT EXISTS Jobs (  
    Id          INTEGER PRIMARY KEY AUTOINCREMENT,  
    Title       TEXT    NOT NULL,  
    Description  TEXT    NOT NULL DEFAULT "",  
    RequiredSkills TEXT  NOT NULL DEFAULT "",  
    Location    TEXT    NOT NULL DEFAULT "",  
    JobType     INTEGER NOT NULL DEFAULT 0, -- 0=Internship, 1=FullTime, 2=PartTime  
    IsActive    INTEGER NOT NULL DEFAULT 1,  
    CreatedAt   TEXT    NOT NULL  
);
```

- **Applications**

```
CREATE TABLE IF NOT EXISTS Applications (  
    Id          INTEGER PRIMARY KEY AUTOINCREMENT,  
    StudentProfileId INTEGER NOT NULL REFERENCES StudentProfiles(Id),  
    JobId       INTEGER NOT NULL REFERENCES Jobs(Id),  
    Status      INTEGER NOT NULL DEFAULT 0, -- 0=Pending, 1=Accepted, 2=Rejected  
    AppliedAt   TEXT    NOT NULL,  
    UpdatedAt   TEXT    NOT NULL,  
    UNIQUE(StudentProfileId, JobId)  
);
```

- **Notifications**

```
CREATE TABLE IF NOT EXISTS Notifications (  
    Id          INTEGER PRIMARY KEY AUTOINCREMENT,  
    UserId      INTEGER NOT NULL REFERENCES Users(Id),  
    Title       TEXT    NOT NULL,  
    Message     TEXT    NOT NULL,  
    NotificationType TEXT  NOT NULL DEFAULT "",  
    ReferenceKey TEXT  NOT NULL DEFAULT "",  
    IsRead      INTEGER NOT NULL DEFAULT 0,  
    CreatedAt   TEXT    NOT NULL  
);
```

4. LINQ Queries

LINQ (Language-Integrated Query) is used throughout the codebase to filter, project, and sort in-memory collections retrieved from the database. The following queries are the most significant.

4.1 Job Search with Skill, Location, and Type Filters

- **Code**

```
IEnumerable<JobSearchResultViewModel> query = jobs.Select(job => new JobSearchResultViewModel  
{  
    Id          = job.Id,
```

```

        Title      = job.Title,
        MatchPercentage = _matchingService.CalculateSkillMatchPercentage(studentSkills, job.RequiredSkills),
        HasApplied    = existingApplications.TryGetValue(job.Id, out var status),
        ApplicationStatus = existingApplications.TryGetValue(job.Id, out var s) ? s.ToString() : string.Empty
    });

    if (!string.IsNullOrEmpty(filters.Skill))
        query = query.Where(j => SearchTextHelper.Contains(j.RequiredSkills, filters.Skill));

    if (!string.IsNullOrEmpty(filters.Location))
        query = query.Where(j => SearchTextHelper.Contains(j.Location, filters.Location));

    if (Enum.TryParse<JobType>(filters.JobType, true, out var jt))
        query = query.Where(j => j.JobType.Equals(jt.ToString(), StringComparison.OrdinalIgnoreCase));

    return query.OrderByDescending(j => j.MatchPercentage).ThenBy(j => j.Title).ToList();

```

- **Explanation**

All active jobs are first projected into `JobSearchResultViewModel` objects. The skill-match percentage and existing-application status are computed inline using the matching service and a dictionary lookup. Three optional filter predicates are then chained conditionally — each only applied when the filter value is non-empty. Finally the results are ordered by descending match percentage (best match first) and then alphabetically by title as a tiebreaker.

4.2 Admin Job Filters (Desktop)

- **Code**

```

IEnumerable<JobListItemViewModel> query = _jobItems;

if (!string.IsNullOrEmpty(searchText))
    query = query.Where(item =>
        SearchTextHelper.Contains(item.Title, searchText) ||
        SearchTextHelper.Contains(item.Location, searchText) ||
        SearchTextHelper.Contains(item.RequiredSkills, searchText));

if (selectedJobType != null && selectedJobType != "All Types")
    query = query.Where(item => item.JobType.Equals(selectedJobType, StringComparison.OrdinalIgnoreCase));

if (selectedActiveStatus != null && selectedActiveStatus != "All Statuses")
    query = query.Where(item => item.ActiveStatus.Equals(selectedActiveStatus, StringComparison.OrdinalIgnoreCase));

```


- **Explanation**

The admin dashboard caches the full job list in `_jobItems` and re-filters in memory on every UI interaction without re-querying the database. The free-text search checks three fields (title, location, skills) using a case-insensitive helper. Two combobox-driven filters for job type and active status are applied only when their value differs from the catch-all sentinel.

4.3 Home ViewModel — Highest Skill Match

- **Code**

```
var highestMatchPercentage = jobs.Count == 0
    ? 0
    : jobs.Max(job => _matchingService.CalculateSkillMatchPercentage(studentSkills, job.RequiredSkills));

var pendingCount = existingApplications
    .Count(application => application.Value == ApplicationStatus.Pending);
```

- **Explanation**

LINQ's `Max()` aggregator finds the single best skill-match percentage across all active jobs for the dashboard headline figure. A null-guard handles the edge case where no jobs exist. `Count()` with a predicate then tallies how many of the student's applications are still pending review.

5. Delegates and Events

The project uses two custom event-based patterns to decouple components and communicate asynchronous outcomes across layers.

5.1 NotificationManager — NotificationCreated Event

- **Purpose**

When an admin changes an application status, the `ApplicationService` calls `NotificationManager` to persist a notification. Instead of the service directly invoking UI code, it raises an event. The desktop form subscribes and shows a `MessageBox`; the web layer can subscribe differently (or ignore it) without modifying the service.

- **Definition (Shared.Services.NotificationManager)**

```
public class NotificationManager
{
    public event EventHandler<Notification>? NotificationCreated;

    public void SendNotification(int userId, string title, string message, ...)
    {
        var notification = new Notification { UserId = userId, Title = title, Message = message, ... };
        // persist to DB ...
        NotificationCreated?.Invoke(this, notification);
    }
}
```

- **Subscription in AdminDashboardForm**

```
_notificationManager.NotificationCreated += NotificationManager_NotificationCreated;

private void NotificationManager_NotificationCreated(object? sender, Notification notification)
{
    MessageBox.Show(notification.Message, notification.Title,
        MessageBoxButtons.OK, MessageBoxIcon.Information);
}
```

- **How it works**

`EventHandler<T>` is a built-in generic delegate. The null-conditional invoke (`?.Invoke`) safely handles the case where no handler is attached. The desktop form wires up its handler in the constructor, so every notification raised by any service that uses this `NotificationManager` instance will pop up immediately in the UI.

5.2 AuditLogger — AdminActionPerformed Event

- **Purpose**

`AuditLogger` is a simple diagnostic utility that records administrator actions. Using an `Action<string>` delegate rather than a full `EventHandler` keeps the class lightweight. The dashboard binds a lambda that prepends a timestamp and inserts the entry at the top of a `ListBox`.

- **Definition (Shared.Utilities.AuditLogger)**

```
public class AuditLogger
{
    public event Action<string>? AdminActionPerformed;

    public void Log(string action)
    {
        AdminActionPerformed?.Invoke(action);
    }
}
```

- **Subscription in AdminDashboardForm**

```
private readonly AuditLogger _auditLogger = new();

// In constructor:
_auditLogger.AdminActionPerformed += action =>
{
    lstAuditLog.Items.Insert(0, $"{DateTime.Now:HH:mm:ss} - {action}");
};

// Usage:
_auditLogger.Log($"Added job: {form.JobResult.Title}");
_auditLogger.Log($"Deleted job: {selectedItem.Title}");
```

- **How it works**

Action<string> is a built-in delegate type that accepts one string argument and returns void. A lambda expression is used as the handler, capturing lstAuditLog from the enclosing form. Items.Insert(0, ...) keeps the newest entry at the top. Every significant admin action — adding, editing, or deleting a job, accepting or rejecting an application — calls _auditLogger.Log() to trigger the event.

6. MVC Structure

The web application (StudentWeb) follows the ASP.NET Core MVC pattern. The three layers are separated to promote testability and maintainability.

6.1 Overview

Layer	Components	Responsibility
Model	JobSearchViewModel, ApplicationListViewModel, HomeViewModel, ProfileEditViewModel	Carry data between Controller and View; validated before processing.
View	Razor .cshtml files (Jobs/Index, Applications/Index, Notifications/Index, Profile/Edit, Account/Login)	Render HTML; use tag helpers and Razor syntax to display model data.
Controller	JobsController, ApplicationsController, NotificationsController, ProfileController, AccountController	Handle HTTP requests, call services, pass models to views or redirect.

6.2 Service Layer

Rather than placing database access in controllers, instead we introduced a service layer between controllers and the database helper:

- **Job Browse Service:** Fetches active jobs, computes skill matches, applies LINQ filters.
- **Student Application Service:** handles application submission with all validation rules.
- **Profile Service:** Updates student profile using an INSERT OR REPLACE pattern.
- **Notification Query Service:** Reads and marks notifications for a given user.
- **Student Auth Service:** Handles registration (with profile creation in a transaction) and login validation.
- **Resume Builder Service:** Fetches the users profile and builds a cv from it.

7. Sources

7.1 GitHub Repository

The complete source code is available at:

<https://github.com/serhatuzunbayir/StudentInternshipSPRING25-26>

7.2 How to Run

Prerequisites

- Visual Studio 2022+ (or VS Code with C# extension)

- .NET 10 SDK
- SQLite (bundled via Microsoft.Data.Sqlite NuGet package — no separate install required)

Steps

Clone the repository

2. Open **StudentInternshipPortal.sln**.
3. Restore packages if necessary: **dotnet restore**
4. Update the database path in the **appsettings.json** files if required.
5. Build the solution: **dotnet build .\StudentInternshipPortal.sln**
6. Start the web application: **dotnet run --project .\StudentWeb\StudentWeb.csproj**
7. Run the Desktop Admin application from Visual Studio.

Default Admin Account

- Username: **admin**
- Password: **admin123**

Students must create their own accounts through the Register page.

7.3 Video Demonstrations

- Desktop Admin Application:
<https://drive.google.com/file/d/1XZRCRuB2jAUzODPGUk4s4hfbekpeHUEv/view?usp=sharing>

8. User Interface and Design

8.1 Desktop Admin Application

The desktop application is built with Windows Forms. The main dashboard is divided into three tabs:

- Jobs Tab — searchable, filterable grid of all job postings with Add, Edit, and Delete buttons.
- Applications Tab — searchable, filterable grid of all applications with Accept, Reject, and Mark Pending buttons.
- Reports Tab — six summary KPI labels (Total Students, Active Jobs, Total Applications, Accepted, Rejected, Pending) plus an Audit Log listbox.

Desktop Admin Dashboard

×

Logged in as: admin

Jobs

Applications

Reports

Add Job

Edit

Delete

Search

Title, location or skills

Type

All Types

▼

Status

All Statuse

▼

Clear

Id	Title	Location	JobType	RequiredSkills	ActiveStatus
2	App Developer	Izmir, Çiğli	Internship	Flutter, C#	Active
1	Software Engi...	Izmir, Bornova	Internship	.Net, C#	Active

Figure 1: Jobs Tab

Desktop Admin Dashboard

×

Logged in as: admin

Jobs

Applications

Reports

Accept

Reject

Reset Status

Search

Student or job

Status

All Statuse

Clear

Refresh

Id	StudentId	StudentName	JobTitle	Status	AppliedAt
2	2	sevval koc	App Developer	Pending	2026-06-02 15:24
1	2	sevval koc	Software Engineer	Pending	2026-06-02 15:24

Figure 2: Applications Tab

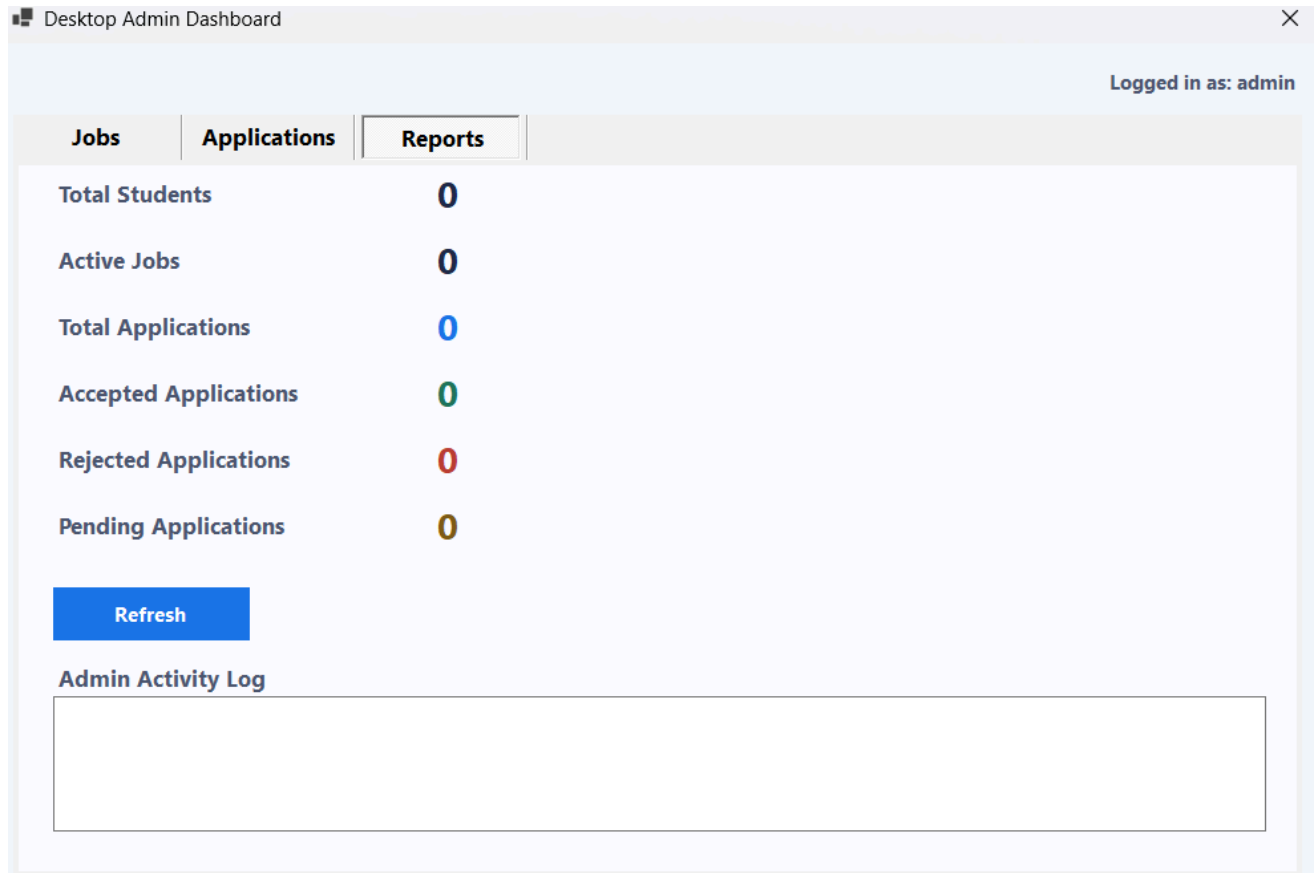


Figure 3: Reports Tab

8.2 Student Web Application

The web application uses ASP.NET Core MVC with Razor views. Key pages include:

- Login / Register: Authentication forms with validation feedback.
- Jobs Index: Card or table layout with skill/location/type filter controls and a match-percentage badge per job.
- My Applications: Status-badged list of all submitted applications.
- Notifications: Notification inbox with Mark as Read and Mark All as Read actions.
- Profile: Editable form for full name, skills (comma-separated), education, experience, phone, and about me.
- CV Builder: Functionality for users to build CV's off of their profiles.

Your Profile

Profile updated successfully!

Full Name

sevval koc

Education

computer enginnering

Phone

+90 5455463686

Skills

C# , .Net

Experience

Internship at fibabanka

About Me

I am a 4th computer enginnering student at IUE.I am looking for a job

Save Profile

Figure 4: Profile Page

Student Jobs

Browse active opportunities

Filter jobs by skill, location, and type. Results are ranked by how well the required skills match your profile.

Skill

Example: C#, SQL, Flutter

Location

Example: Istanbul, Remote

Job Type

All Types

Search

Clear

Results

2 job(s) found

Software Engineer

100% match

Izmir, Bornova • Internship

.Net Development

REQUIRED SKILLS

.Net, C#

Apply now

App Developer

50% match

Izmir, Çiğli • Internship

Flutter development

REQUIRED SKILLS

Flutter, C#

Apply now

Figure 5: Available Jobs

Applications

2 application(s)

App Developer

Pending

Izmir, Çiğli | Internship

Applied on 2026-06-02 15:24

REQUIRED SKILLS

Flutter, C#

Software Engineer

Pending

Izmir, Bornova | Internship

Applied on 2026-06-02 15:24

REQUIRED SKILLS

.Net, C#

Figure 6: User's Applications

Resume Customizer

Design and customize your resume in real-time.

1. TYPOGRAPHY

Font Family

Sans

Serif

Mono

2. PERSONAL INFORMATION

Full Name

seval koc

Job Title

Student

Address

Izmir, Turkey

Email Address

seval

Phone Number

+90 5455463686

3. CONTENT SECTIONS

SEVVAL KOC

STUDENT

Izmir, Turkey • +90 5455463686 • seval

SUMMARY

I am a 4th computer engineering student at IUE.I am looking for a job

EXPERIENCE

Internship at fibabanka

EDUCATION

computer engineering

SKILLS

C# • .Net • java

Figure 7: Resume Builder

Inbox

2 unread of 2 notification(s)

Mark all as read

Application Rejected

02.06.2026 15:25

Your application for 'Software Engineer' has been rejected.

Mark as read

Application Accepted

02.06.2026 15:25

Good news. Your application for 'App Developer' has been accepted.

Mark as read

Application Rejected

02.06.2026 15:25

Your application for 'Software Engineer' has been rejected.

Application Rejected

Application Accepted

02.06.2026 15:25

Good news. Your application for 'App Developer' has been accepted.

Application Accepted

Figure 8: Notifications

9. Unit Tests

The following 10 unit test cases cover the core business logic of the application. The tests are written using xUnit and located in the **PortalTests** project. These tests mainly focus on two shared helper/service classes: *MatchingService* and *SearchTextHelper*.

MatchingService is tested to verify that the skill matching percentage between a student's skills and a job's required skills is calculated correctly. *SearchTextHelper* is tested to verify that text searching works correctly in a case-insensitive way.

#	Test Name	Description	Expected	Result
1	CalculateSkillMatchPercentage_FullMatch_Returns100	Checks whether the method returns 100 when the student has all required skills.	Returns 100	PASS
2	CalculateSkillMatchPercentage_NoMatch_Returns0	Checks whether the method returns 0 when the student has none of the required skills.	Returns 0	PASS
3	CalculateSkillMatchPercentage_PartialMatch_ReturnsCorrectPercentage	Checks whether the method correctly calculates a partial match percentage.	Returns 50	PASS
4	CalculateSkillMatchPercentage_CaseInsensitivity_Returns100	Checks whether skill comparison is case-insensitive.	Returns 100	PASS
5	CalculateSkillMatchPercentage_EmptyRequiredSkills_Returns0	Checks whether the method returns 0 when the required skills field is empty.	Returns 0	PASS
6	CalculateSkillMatchPercentage_ExtraStudentSkills_Returns100	Checks whether extra student skills do not reduce the score when all required skills are matched.	Returns 100	PASS
7	CalculateSkillMatchPercentage_TrimAndWhiteSpace_ReturnsCorrectPercentage	Checks whether spaces around skill names are trimmed and handled correctly.	Returns 100	PASS
8	Contains_CaseInsensitiveSearch_ReturnsTrue	Checks whether the search helper can find a word regardless of letter casing.	Returns true	PASS

#	Test Name	Description	Expected	Result
9	Contains_EmptyValue_ReturnsTrue	Checks whether searching with an empty value returns true..	Returns true	PASS
10	Contains_Mismatch_ReturnsFalse	Checks whether the helper returns false when the searched word does not exist in the source text.	Returns false	PASS

9.1 Sample Test Scripts

- **Test 1 - CalculateSkillMatchPercentage_FullMatch_Returns100**

```
[Fact]
public void CalculateSkillMatchPercentage_FullMatch_Returns100()
{
    // Arrange
    string studentSkills = "C#, SQL, ASP.NET Core";
    string requiredSkills = "C#, SQL";

    // Act
    int result = _matchingService.CalculateSkillMatchPercentage(studentSkills, requiredSkills);

    // Assert
    Assert.Equal(100, result);
}
```

- **Test 2 - RegisterStudent_Success**

```
[Fact]
public void CalculateSkillMatchPercentage_NoMatch_Returns0()
{
    // Arrange
    string studentSkills = "Java, Python";
    string requiredSkills = "C#, SQL";

    // Act
    int result = _matchingService.CalculateSkillMatchPercentage(studentSkills, requiredSkills);

    // Assert
    Assert.Equal(0, result);
}
```

- **Test 3 - CalculateSkillMatchPercentage_PartialMatch_ReturnsCorrectPercentage**

[Fact]

```
public void CalculateSkillMatchPercentage_PartialMatch_ReturnsCorrectPercentage()
{
    // Arrange
    string studentSkills = "C#, Python, HTML";
    string requiredSkills = "C#, SQL, CSS, HTML"; // 2 of 4 match (50%)

    // Act
    int result = _matchingService.CalculateSkillMatchPercentage(studentSkills, requiredSkills);

    // Assert
    Assert.Equal(50, result);
}
```

- **Test 4 - CalculateSkillMatchPercentage_CaseInsensitivity_Returns100**

[Fact]

```
public void CalculateSkillMatchPercentage_CaseInsensitivity_Returns100()
{
    // Arrange
    string studentSkills = "c#, sql";
    string requiredSkills = "C#, SQL";

    // Act
    int result = _matchingService.CalculateSkillMatchPercentage(studentSkills, requiredSkills);

    // Assert
    Assert.Equal(100, result);
}
```

- **Test 5 - CalculateSkillMatchPercentage_EmptyRequiredSkills_Returns0**

[Fact]

```
public void CalculateSkillMatchPercentage_EmptyRequiredSkills_Returns0()
{
    // Arrange
    string studentSkills = "C#, SQL";
    string requiredSkills = "";

    // Act
    int result = _matchingService.CalculateSkillMatchPercentage(studentSkills, requiredSkills);

    // Assert
    Assert.Equal(0, result);
}
```

```
}
```

- **Test 6 - CalculateSkillMatchPercentage_ExtraStudentSkills_Returns100**

[Fact]

```
public void CalculateSkillMatchPercentage_ExtraStudentSkills_Returns100()
{
    // Arrange
    string studentSkills = "C#, SQL, Python, Java, Javascript";
    string requiredSkills = "C#";

    // Act
    int result = _matchingService.CalculateSkillMatchPercentage(studentSkills, requiredSkills);

    // Assert
    Assert.Equal(100, result);
}
```

- **Test 7 - CalculateSkillMatchPercentage_TrimAndWhitespace_ReturnsCorrectPercentage**

[Fact]

```
public void CalculateSkillMatchPercentage_TrimAndWhitespace_ReturnsCorrectPercentage()
{
    // Arrange
    string studentSkills = " C# , SQL ";
    string requiredSkills = "C#,SQL";

    // Act
    int result = _matchingService.CalculateSkillMatchPercentage(studentSkills, requiredSkills);

    // Assert
    Assert.Equal(100, result);
}
}
```

- **Test 8 - Contains_CaseInsensitiveSearch_ReturnsTrue**

[Fact]

```
public void Contains_CaseInsensitiveSearch_ReturnsTrue()
{
    // Arrange
    string source = "Web Developer Position";
    string value = "DEVELOPER";
```

```
// Act
bool result = SearchTextHelper.Contains(source, value);

// Assert
Assert.True(result);
}
```

- **Test 9 - Contains_EmptyValue_ReturnsTrue**

```
[Fact]
public void Contains_EmptyValue_ReturnsTrue()
{
    // Arrange
    string source = "Any text";
    string value = "";

    // Act
    bool result = SearchTextHelper.Contains(source, value);

    // Assert
    Assert.True(result);
}
```

- **Test 10 - Contains_Mismatch_ReturnsFalse**

```
[Fact]
public void Contains_Mismatch_ReturnsFalse()
{
    // Arrange
    string source = "C#, SQL, HTML";
    string value = "Python";

    // Act
    bool result = SearchTextHelper.Contains(source, value);

    // Assert
    Assert.False(result);
}
```

9.2 Test Run Execution Output

The unit tests were executed using the **dotnet test** command. The test run completed successfully with no failed or skipped tests.

Başarılı! - Başarısız: 0, Başarılı: 10, Atlanan: 0, Toplam: 10

This result confirms that all 10 implemented unit tests passed successfully.

10. Conclusion

The Student Internship Portal successfully demonstrates a full-stack .NET solution that integrates a Windows Forms desktop application for administrators and an ASP.NET Core MVC web application for students, both sharing a common business logic and data access layer. The project implements key C# concepts — LINQ for in-memory querying and filtering, delegates and events for decoupled cross-layer communication, and the MVC architectural pattern for clean separation of concerns in the web tier. The SQLite-backed persistence layer keeps the setup friction-free while parameterised queries ensure security. Password hashing via ASP.NET Identity, anti-forgery token validation, and role-based authorization further reinforce the security posture. The ten unit tests verify the most critical user-facing scenarios and provide a regression safety net for future development. Overall, the project achieves its stated objectives of connecting students with career opportunities, empowering admins with efficient management tools, and showcasing modern C# engineering practices in a cohesive, maintainable codebase.